# FRONT-END DEVELOPMENT HACKATHON Level-2

## Activity Tracker with Real-Time Progress


STUDENT NAME : AKASH R
YEAR : II-YEAR
COLLEGE CODE : bru1021008
COLLEGE NAME : GOVERNMENT ARTS AND SCIENCE COLLEGE SATHYAMANGALAM
DEPARTMENT : BACHELOR OF COMPUTER SCIENCE
SEMESTER : IV-SEMESTER
REG NO : 2422K1433
NMID : C6FC36EF208EEE4A78A48EB5ABDDB616

DATE : 10/04/2026
TITLE : Activity Tracker with Real-Time Progress
GITHUB LINK:

### 1. Objective

The objective of this project is to implement front-end interactivity to track activity status and display real-time progress using JavaScript. It helps users mark tasks as completed and dynamically updates the UI without refreshing the page.

### 2. Problem Focus

In Level-1, the UI was static with no interactivity. In Level-2, the focus is to convert the static interface into a dynamic one by adding event handling, state changes, and real-time updates.

### 3. Application Overview

This application is an Activity Tracker that allows users to view tasks, mark them as completed, and track progress in real-time.

### 4. Data Handling Approach

Data is managed using a JavaScript array. Local Storage can be used optionally to store user progress.

### 5. UI Components Design

Includes Activity List, Checkboxes/Buttons, and Progress Display section.

### 6. Interactivity Implementation

Uses JavaScript event listeners to update task status and UI dynamically.

### 7. Application Logic

Counts total and completed tasks, calculates percentage, and updates UI instantly.

### 8. Technology Used

HTML, CSS, JavaScript

### 9. Best Practices Followed

Meaningful naming, no hardcoding, comments added, edge cases handled.

### 10. Key Features Implemented

Task completion, real-time progress tracking, dynamic updates.

### 11. Expected Output / Deliverables

Working app, interactive tracker, progress display, GitHub repo.

### 12. Learning Outcome

Learned event handling, DOM manipulation, and interactive UI development.

### 13. Conclusion

This project successfully converts a static user interface into an interactive Activity Tracker. Users can mark tasks as completed, and the application updates progress in real-time without page reload. By using JavaScript for event handling and DOM manipulation, the app becomes dynamic and user-friendly. Overall, this project enhances user experience and demonstrates practical front-end development skills.

### 14. Coding

**HTML**

```
<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <title>Activity Tracker</title>

 <link rel="stylesheet" href="style.css">
```

```
</head>
<body>
<div class="container">
  <h1>Activity Tracker</h1>
 <p id="progress">0 out of 0 completed</p>
<ul id="activityList"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

**CSS**

```
body {
  font-family: Arial, sans-serif;
  background-color: yellow;
  text-align: center;
}
.container {
  width: 400px;
  margin: 50px auto;
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 0 10px gray;
}
```

```
h1 {
  color: #333;
}
ul {
  list-style: none;
  padding: 0;
}
li {
  background: #eaeaea;
  margin: 10px 0;
  padding: 10px;
  border-radius: 5px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
button {
  padding: 5px 10px;
  border: none;
  background-color: green;
  color: white;
  border-radius: 5px;
  cursor: pointer;
}
```

```
button:hover {
  background-color: darkgreen;
}
.completed {
  text-decoration: line-through;
  color: gray;
}
```

**JAVA SCRIPT**

```
// Activity Data
let activities = [
  { name: "HTML Practice", completed: false },
  { name: "CSS Styling", completed: false },
  { name: "JavaScript Learning", completed: false },
  { name: "Mini Project", completed: false }
];
// Render Activities
function renderActivities() {
  const list = document.getElementById("activityList");
  list.innerHTML = "";
 let completedCount = 0;
 activities.forEach((activity, index) => {
    const li = document.createElement("li");
 if (activity.completed) {
      li.classList.add("completed");
```

```
    completedCount++;
  }
li.innerHTML = `
    ${activity.name}
    <button onclick="markComplete(${index})">
     ${activity.completed ? "Done" : "Complete"}
    </button>
  `;
list.appendChild(li);
 });
 // Update Progress
 document.getElementById("progress").innerText =
  `${completedCount} out of ${activities.length} completed`;
// All Completed Message
 if (completedCount === activities.length) {
  document.getElementById("progress").innerText += " 🎉 All Activities Completed!";
 }
}
// Mark Complete Function
function markComplete(index) {
 activities[index].completed = true;
 renderActivities();
}
// Initial Load
```

renderActivities();

## 15. Output